[Nom de l'établissement / Université]

[Département / Filière]

Compte rendu du TP2

Documentation et tests unitaires d'un package Python

Package `simulateur` : simulation de trafic routier

Réalisé par : [Prénom NOM]

Encadré par : [Nom de l'enseignant]

Année universitaire : 2025–2026

Novembre 2026

Contents

Introduction **4**

1 Le package `simulateur` **5**
1.1 Organisation 5
1.2 Rôle de chaque module 6
1.3 Règles du simulateur 6

2 Documentation **8**
2.1 Docstrings 8
2.2 Génération de la documentation avec Sphinx 9
2.3 Fichier README.md 11

3 Conception des tests unitaires **12**
3.1 Méthode 12
3.2 Les quatre types de tests 13
3.3 Tests de la simulation 13

4 Exécution et analyse des résultats **15**
4.1 Commandes 15
4.2 Avant correction (code d'origine) 15
4.3 Après correction 16
4.4 Absence de régression 20

5 Anomalies détectées et corrections **21**
5.1 Défauts de comportement 21
5.2 Validations manquantes 24
5.3 Choix de conception 25

Conclusion **26**

List of Figures

1.1 Arborescence réelle du projet 6

2.1 Documentation lue avec `help()` 9
2.2 Docstring de module affichée 9
2.3 Compilation de la documentation avec `sphinx-build` 10
2.4 Page d'accueil de la documentation HTML 10
2.5 Référence de l'API générée depuis les docstrings 11
2.6 Exemple d'utilisation du package 11

4.1 Exécution sur le code d'origine : 28 échecs, 66 réussites, 1 test bloqué . . . 16
4.2 `pytest -v` après correction : 95 tests réussis 17
4.3 Tests du module `vehicule` 18
4.4 Tests du module `route` 19
4.5 Tests du module `simulation` 20
4.6 Résumé final 20

5.1 Message d'erreur clair après correction (`Route` avec longueur négative) . . . 25

List of Tables

1.1 Modules du package 6

2.1 Fichiers de la documentation Sphinx 9

3.1 Organisation des tests par type et par module 13

4.1 Résultats avant correction 15
4.2 Résultats après correction 16

5.1 Anomalies de comportement 21
5.2 Validations ajoutées 24

Introduction

Le package `simulateur` simule le déplacement de véhicules sur un réseau routier modélisé par un graphe. Avant de le distribuer, ce TP demande de **documenter** le code, de rédiger un **README**, de mettre en place une suite de **tests unitaires avec pytest**, puis de **corriger les anomalies** que ces tests révèlent.

Ce compte rendu présente successivement :

1. l'organisation du package et le travail de documentation (docstrings, README, site Sphinx) ;
2. la conception des tests : cas nominaux, cas limites, cas invalides et cohérence ;
3. l'exécution des tests avant et après correction, avec le décompte des tests réussis et échoués ;
4. la liste des anomalies détectées et des corrections effectuées.

Chapter 1

Le package `simulateur`

1.1 Organisation

Le code initial, réparti en plusieurs dossiers, a été réorganisé selon la structure demandée par l'énoncé. Un module interne `_validation.py` a été ajouté pour centraliser les contrôles de paramètres (même type d'exception partout, sans duplication de code).

Listing 1.1: Structure du projet

```
simulateur/
|-- simulateur/
| |-- __init__.py API publique du package
| |-- _validation.py contrôles de type et de valeur (interne)
| |-- vehicule.py classe Vehicule
| |-- route.py classe Route
| `-- simulation.py ReseauRoutier, Analyseur, Simulation
|-- tests/
| |-- conftest.py fixtures partagées
| |-- test_vehicule.py
| |-- test_route.py
| `-- test_simulation.py
|-- docs/ documentation Sphinx (conf.py, *.rst)
|-- data/config_reseau.json
|-- exemples/exemple_simple.py
|-- README.md
`-- pyproject.toml
```



Figure 1.1: Arborescence réelle du projet

1.2 Rôle de chaque module

Module	Classes	Rôle
`vehicule.py`	`Vehicule`	Itinéraire, position, vitesse ; avance pendant un pas de temps et change de route
`route.py`	`Route`	Tronçon orienté : capacité, occupation, vitesse autorisée, accueil des véhicules
`simulation.py`	`ReseauRoutier`, `Analyseur`, `Simulation`	Graphe et Dijkstra ; statistiques et congestion ; boucle de simulation et lecture de la configuration
`_validation.py`	(fonctions)	Validation de nombres, d'entiers et de textes

Table 1.1: Modules du package

1.3 Règles du simulateur

Les tests s'appuient sur les règles suivantes.

- Capacité d'une route : $\max\big(1,\ \text{longueur} \times \text{nb_voies}/25\big)$; occupation = véhicules / capacité.
- Vitesse autorisée : $v = v_{\text{limite}} \times \max(0{,}1,\ 1 - \text{occupation})$.
- Vitesse réelle d'un véhicule : $\min(v_{\max},\ v)$; distance $= \dfrac{v}{3{,}6}\Delta t$.
- Itinéraire : chemin au temps de parcours théorique minimal (Dijkstra).

- Un véhicule se trouve toujours sur la route prévue par son itinéraire, à une position comprise entre 0 et la longueur de la route.

Chapter 2

Documentation

2.1 Docstrings

Tous les éléments du package sont documentés : **49 modules, classes, fonctions et méthodes publics** (vérifié automatiquement avec le module `ast`, aucun manquant). Le format retenu est le style Google :

- **module** : objectif du fichier ;
- **classe** : rôle, responsabilités et liste des attributs avec leur type ;
- **fonction / méthode** : objectif, `Args` (types), `Returns`, `Raises`.

Les commentaires dans le code ont été conservés uniquement là où un point mérite explication (par exemple la conversion km/h → m/s ou le report de la partie fractionnaire de véhicules à créer).

Listing 2.1: Exemple de docstring complète (`route.py`)

```
def mettre_a_jour_vehicules(self, delta_t):
    """Fait avancer tous les véhicules de la route pendant ``delta_t`` secondes.

    La vitesse autorisée est calculée une seule fois, avec la charge du début de pas.

    Args:
        delta_t (float): Durée du pas en secondes, positive ou nulle.

    Returns:
        list[Vehicule]: Véhicules ayant atteint ou dépassé la fin de la route.

    Raises:
        TypeError: Si ``delta_t`` n'est pas un nombre.
        ValueError: Si ``delta_t`` est négatif.
    """
```

```
$ python3 -c "from simulateur import Route; help(Route.vitesse_autorisee)"

Help on function vitesse_autorisee in module simulateur.route:

vitesse_autorisee(self)
    Calcule la vitesse praticable vu l'encombrement.

    Formule : ``limite * max(FACTEUR_VITESSE_MIN, 1 - occupation)``.

    Returns:
        float: Vitesse autorisée en km/h, comprise entre 10 % de la limite et la limite.
```

Figure 2.1: Documentation lue avec `help()`

```
$ python3 -c "import simulateur.route as m; print(m.__doc__)"

Modèle d'une route : tronçon à sens unique entre deux intersections.

La vitesse praticable sur une route diminue avec son occupation (modèle linéaire
de type Greenshields).
```

Figure 2.2: Docstring de module affichée

2.2 Génération de la documentation avec Sphinx

Les docstrings ne sont pas seulement lisibles dans le code : **Sphinx** les transforme en un site HTML navigable (référence de l'API, index, recherche). La documentation est dans le dossier `docs/`.

Fichier	Rôle
`docs/conf.py`	Configuration : chemin du package, extensions, thème, langue française
`docs/index.rst`	Page d'accueil et sommaire (`toctree`)
`docs/utilisation.rst`	Installation, exemples, erreurs levées
`docs/api.rst`	Référence de l'API, générée avec `automodule`
`docs/tests.rst`	Commandes de test
`docs/Makefile`, `make.bat`	Commande `make html` (Linux/macOS, Windows)

Table 2.1: Fichiers de la documentation Sphinx

Trois extensions sont activées dans `conf.py` :

- `sphinx.ext.autodoc` : extrait la documentation des docstrings ;
- `sphinx.ext.napoleon` : comprend le style Google (`Args`, `Returns`, `Raises`) utilisé dans le package ;
- `sphinx.ext.viewcode` : ajoute un lien vers le code source de chaque fonction.

Listing 2.2: Extrait de `docs/api.rst`

```
Module ‘‘route‘‘
---------------

.. automodule:: simulateur.route
   :members:
```

Listing 2.3: Génération de la documentation

```
pip install -e ".[docs]"
cd docs
make html # Windows : make.bat html
# ouvrir docs/_build/html/index.html
```

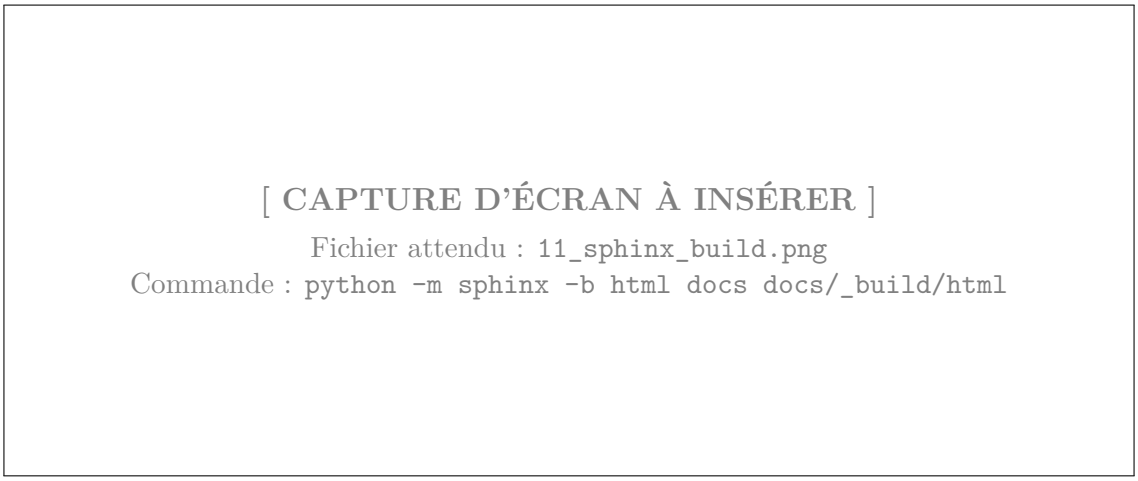


Figure 2.3: Compilation de la documentation avec `sphinx-build`



Figure 2.4: Page d'accueil de la documentation HTML

[CAPTURE D'ÉCRAN À INSÉRER]
Fichier attendu : `13_sphinx_api.png`
Commande : `ouvrir docs/_build/html/api.html`

Figure 2.5: Référence de l'API générée depuis les docstrings

2.3 Fichier README.md

Le `README.md` contient : la présentation du package, ses fonctionnalités, les prérequis (Python $\geq$ 3.9, aucune dépendance d'exécution), la procédure d'installation (`pip install -e ".[dev]"`), un exemple d'utilisation, le format du fichier de configuration, les exceptions levées et les commandes de test. Une section du README explique aussi comment générer la documentation Sphinx. Un fichier `pyproject.toml` décrit le package et la configuration de pytest.

Listing 2.4: Installation et lancement des tests

```
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
pytest -v
```



Figure 2.6: Exemple d'utilisation du package

Chapter 3

Conception des tests unitaires

3.1 Méthode

Les tests ont été écrits *à partir des spécifications de l'énoncé* puis exécutés d'abord sur le code d'origine. Chaque échec ainsi observé révèle donc une validation manquante ou un défaut réel, corrigé ensuite dans le package. Un même fichier de tests sert avant et après correction, ce qui permet de mesurer l'effet des corrections et de vérifier l'absence de régression.

3.2 Les quatre types de tests

Type	Module	Exemples de tests
Cas nominal	`vehicule`	création, conversion km/h → m/s (3 jeux de données), temps de parcours, changement de route
	`route`	capacité et occupation, ajout/retrait d'un véhicule, véhicules arrivés en bout de route
	`simulation`	Dijkstra, construction depuis la configuration, simulation de 30 tours, génération selon le débit
Cas limite	`vehicule`	vitesse nulle, durée nulle, plafonnement par $v_{\max}$, itinéraire d'une seule route
	`route`	route vide, pleine (40/40) et surchargée (60/40), capacité minimale de 1, plancher de vitesse à 10 %
	`simulation`	0 tour, débit nul, chemin vers soi-même, trajet impossible
Cas invalide	`vehicule`	$v_{\max} \leq 0$, itinéraire vide, départ négatif, identifiant vide, mauvais type, $\Delta t < 0$
	`route`	longueur ou limite ≤ 0, voies $\in \{0, -1, 1{,}5\}$, origine = destination, nom vide, mauvais type
	`simulation`	`n_tours`< 0, `delta_t`≤ 0, débit négatif, plage de vitesse incohérente, fichier absent, doublons, intersection inconnue
Cohérence	`vehicule`	vitesse $\leq \min(v_{\max}, v_{\text{auto}})$, position jamais décroissante
	`route`	vitesse décroissante avec la charge, toujours entre 10 % et 100 % de la limite
	`simulation`	conservation des véhicules, reproductibilité (graine), position dans $[0, \text{longueur}]$ à chaque tour, itinéraire respecté

Table 3.1: Organisation des tests par type et par module

3.3 Tests de la simulation

Les quatre points demandés par l'énoncé sont couverts :

- **exécution valide** : `test_simulation_nominale` (30 tours, historique de 30 entrées, temps simulé 1800 s) ;
- **paramètres invalides** : `n_tours`, `delta_t`, débit, plage de vitesse, fichier ;
- **règles de déplacement** : `test_vehicules_evoluent_selon_les_regles_de_deplacement` (progression sur la route, vitesse $\leq$ limite et $\leq v_{\max}$) ;
- **contraintes du réseau** : `test_contraintes_du_reseau_respectees_a_chaque_tour` (60 tours, position dans la route, itinéraire respecté, aucun doublon).

Listing 3.1: Fixtures et test paramétré (`tests/`)

```python
@pytest.fixture
def reseau_abc():
    r = ReseauRoutier()
    for nom, x in (("A", 0), ("B", 1), ("C", 2)):
        r.ajouter_intersection(nom, x, 0)
    r.ajouter_route(Route("A-B", "A", "B", 1000, 50))
    r.ajouter_route(Route("B-C", "B", "C", 1000, 50))
    return r

@pytest.mark.parametrize("nb_veh, attendue", [
    (0, 50.0), (10, 37.5), (20, 25.0), (40, 5.0), (80, 5.0)])
def test_vitesse_selon_occupation(nb_veh, attendue):
    r = Route("A-B", "A", "B", 1000, 50)
    remplir(r, nb_veh)
    assert r.vitesse_autorisee() == pytest.approx(attendue)

@pytest.mark.parametrize("longueur", [0, -100])
def test_longueur_non_positive_refusee(longueur):
    with pytest.raises(ValueError):
        Route("A-B", "A", "B", longueur, 50)
```

Chapter 4

Exécution et analyse des résultats

4.1 Commandes

```
pytest -v # tous les tests
pytest tests/test_vehicule.py -v # un module
pytest tests/test_route.py -v
pytest tests/test_simulation.py -v
```

4.2 Avant correction (code d'origine)

Les 95 tests ont été lancés sur le code d'origine réorganisé mais non modifié. Un test ne se termine jamais (boucle infinie, anomalie S1) : il a dû être interrompu par un délai maximal.

Module	Tests	Réussis	Échoués	Bloqués
`test_vehicule.py`	25	17	8	0
`test_route.py`	28	17	11	0
`test_simulation.py`	42	32	9	1
Total	**95**	**66**	**28**	**1**

Table 4.1: Résultats avant correction

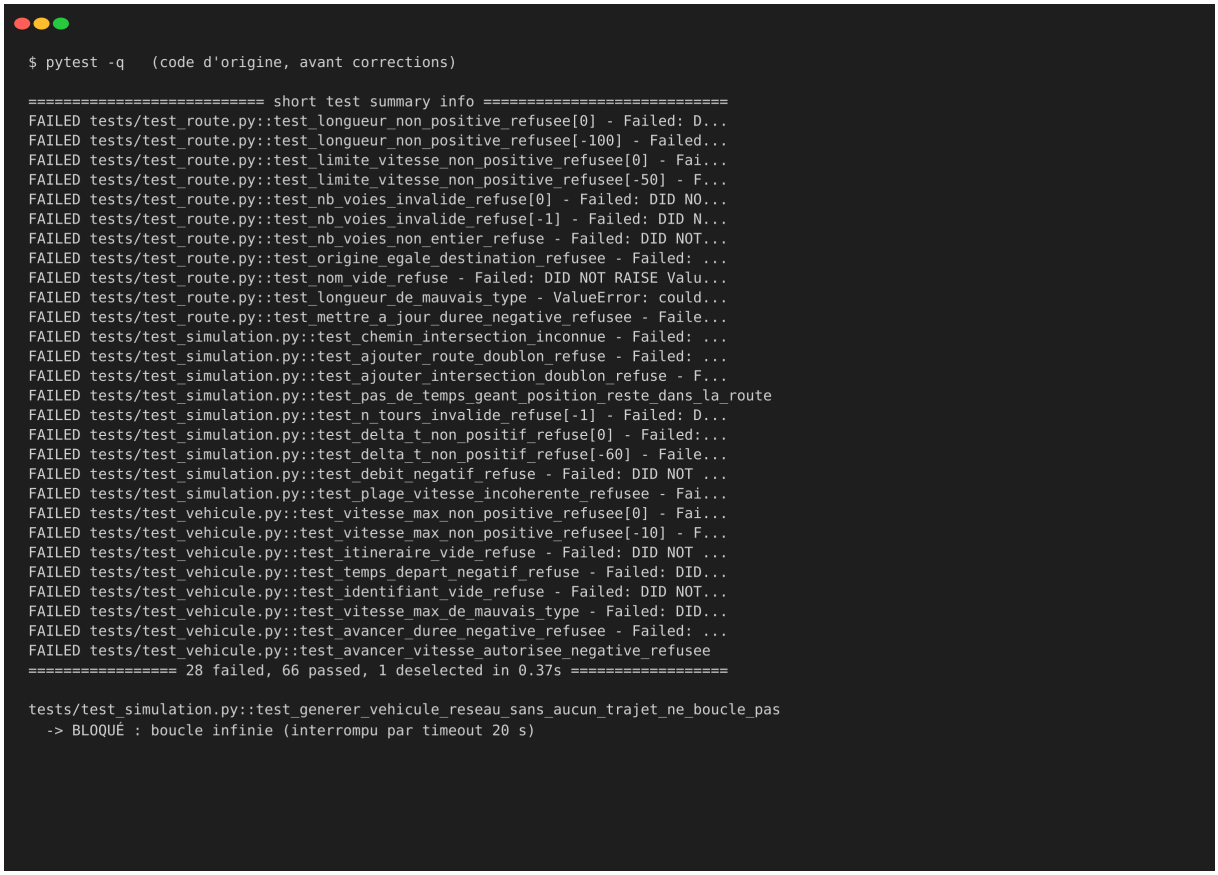


Figure 4.1: Exécution sur le code d'origine : 28 échecs, 66 réussites, 1 test bloqué

4.3 Après correction

Module	Tests	Réussis	Échoués
`test_vehicule.py`	25	25	0
`test_route.py`	28	28	0
`test_simulation.py`	42	42	0
Total	**95**	**95**	**0**

Table 4.2: Résultats après correction

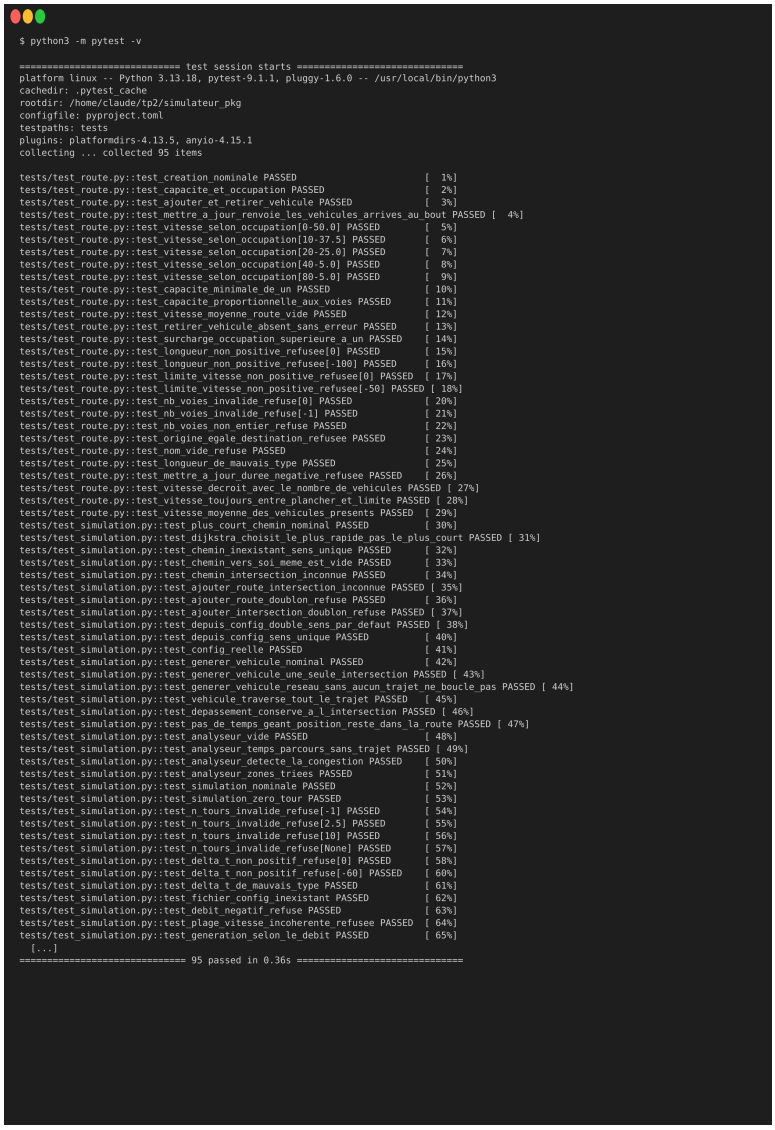


Figure 4.2: `pytest -v` après correction : 95 tests réussis

```
$ python3 -m pytest tests/test_vehicule.py -v

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/tp2/simulateur_pkg
configfile: pyproject.toml
plugins: platformdirs-4.13.5, anyio-4.15.1
collecting ... collected 25 items

tests/test_vehicule.py::test_creation_nominale PASSED                    [  4%]
tests/test_vehicule.py::test_avancer_convertit_kmh_en_metres[36-60-600] PASSED [  8%]
tests/test_vehicule.py::test_avancer_convertit_kmh_en_metres[72-30-600] PASSED [ 12%]
tests/test_vehicule.py::test_avancer_convertit_kmh_en_metres[3.6-10-10] PASSED [ 16%]
tests/test_vehicule.py::test_avancer_cumule_la_distance PASSED           [ 20%]
tests/test_vehicule.py::test_temps_parcours PASSED                       [ 24%]
tests/test_vehicule.py::test_nom_route_suivante PASSED                   [ 28%]
tests/test_vehicule.py::test_changer_de_route PASSED                     [ 32%]
tests/test_vehicule.py::test_vitesse_plafonnee_par_vitesse_max PASSED    [ 36%]
tests/test_vehicule.py::test_avancer_vitesse_nulle PASSED                [ 40%]
tests/test_vehicule.py::test_avancer_duree_nulle PASSED                  [ 44%]
tests/test_vehicule.py::test_itineraire_un_seul_troncon PASSED           [ 48%]
tests/test_vehicule.py::test_itineraire_est_copie PASSED                 [ 52%]
tests/test_vehicule.py::test_vitesse_max_non_positive_refusee[0] PASSED  [ 56%]
tests/test_vehicule.py::test_vitesse_max_non_positive_refusee[-10] PASSED [ 60%]
tests/test_vehicule.py::test_itineraire_vide_refuse PASSED               [ 64%]
tests/test_vehicule.py::test_temps_depart_negatif_refuse PASSED          [ 68%]
tests/test_vehicule.py::test_identifiant_vide_refuse PASSED              [ 72%]
tests/test_vehicule.py::test_vitesse_max_de_mauvais_type PASSED          [ 76%]
tests/test_vehicule.py::test_avancer_duree_negative_refusee PASSED       [ 80%]
tests/test_vehicule.py::test_avancer_vitesse_autorisee_negative_refusee PASSED [ 84%]
tests/test_vehicule.py::test_vitesse_jamais_superieure_aux_deux_limites[50-100] PASSED [ 88%]
tests/test_vehicule.py::test_vitesse_jamais_superieure_aux_deux_limites[130-20] PASSED [ 92%]
tests/test_vehicule.py::test_vitesse_jamais_superieure_aux_deux_limites[90-90] PASSED [ 96%]
tests/test_vehicule.py::test_position_ne_decroit_jamais PASSED           [100%]

============================== 25 passed in 0.03s ==============================
```

Figure 4.3: Tests du module `vehicule`

```
$ python3 -m pytest tests/test_route.py -v

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/tp2/simulateur_pkg
configfile: pyproject.toml
plugins: platformdirs-4.13.5, anyio-4.15.1
collecting ... collected 28 items

tests/test_route.py::test_creation_nominale PASSED                       [  3%]
tests/test_route.py::test_capacite_et_occupation PASSED                  [  7%]
tests/test_route.py::test_ajouter_et_retirer_vehicule PASSED             [ 10%]
tests/test_route.py::test_mettre_a_jour_renvoie_les_vehicules_arrives_au_bout PASSED [ 14%]
tests/test_route.py::test_vitesse_selon_occupation[0-50.0] PASSED        [ 17%]
tests/test_route.py::test_vitesse_selon_occupation[10-37.5] PASSED       [ 21%]
tests/test_route.py::test_vitesse_selon_occupation[20-25.0] PASSED       [ 25%]
tests/test_route.py::test_vitesse_selon_occupation[40-5.0] PASSED        [ 28%]
tests/test_route.py::test_vitesse_selon_occupation[80-5.0] PASSED        [ 32%]
tests/test_route.py::test_capacite_minimale_de_un PASSED                 [ 35%]
tests/test_route.py::test_capacite_proportionnelle_aux_voies PASSED      [ 39%]
tests/test_route.py::test_vitesse_moyenne_route_vide PASSED              [ 42%]
tests/test_route.py::test_retirer_vehicule_absent_sans_erreur PASSED     [ 46%]
tests/test_route.py::test_surcharge_occupation_superieure_a_un PASSED    [ 50%]
tests/test_route.py::test_longueur_non_positive_refusee[0] PASSED        [ 53%]
tests/test_route.py::test_longueur_non_positive_refusee[-100] PASSED     [ 57%]
tests/test_route.py::test_limite_vitesse_non_positive_refusee[0] PASSED  [ 60%]
tests/test_route.py::test_limite_vitesse_non_positive_refusee[-50] PASSED [ 64%]
tests/test_route.py::test_nb_voies_invalide_refuse[0] PASSED             [ 67%]
tests/test_route.py::test_nb_voies_invalide_refuse[-1] PASSED            [ 71%]
tests/test_route.py::test_nb_voies_non_entier_refuse PASSED              [ 75%]
tests/test_route.py::test_origine_egale_destination_refusee PASSED       [ 78%]
tests/test_route.py::test_nom_vide_refuse PASSED                         [ 82%]
tests/test_route.py::test_longueur_de_mauvais_type PASSED                [ 85%]
tests/test_route.py::test_mettre_a_jour_duree_negative_refusee PASSED    [ 89%]
tests/test_route.py::test_vitesse_decroit_avec_le_nombre_de_vehicules PASSED [ 92%]
tests/test_route.py::test_vitesse_toujours_entre_plancher_et_limite PASSED [ 96%]
tests/test_route.py::test_vitesse_moyenne_des_vehicules_presents PASSED  [100%]

============================== 28 passed in 0.04s ==============================
```

Figure 4.4: Tests du module `route`

```
$ python3 -m pytest tests/test_simulation.py -v

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/tp2/simulateur_pkg
configfile: pyproject.toml
plugins: platformdirs-4.13.5, anyio-4.15.1
collecting ... collected 42 items

tests/test_simulation.py::test_plus_court_chemin_nominal PASSED          [  2%]
tests/test_simulation.py::test_dijkstra_choisit_le_plus_rapide_pas_le_plus_court PASSED [  4%]
tests/test_simulation.py::test_chemin_inexistant_sens_unique PASSED      [  7%]
tests/test_simulation.py::test_chemin_vers_soi_meme_est_vide PASSED      [  9%]
tests/test_simulation.py::test_chemin_intersection_inconnue PASSED       [ 11%]
tests/test_simulation.py::test_ajouter_route_intersection_inconnue PASSED [ 14%]
tests/test_simulation.py::test_ajouter_route_doublon_refuse PASSED       [ 16%]
tests/test_simulation.py::test_ajouter_intersection_doublon_refuse PASSED [ 19%]
tests/test_simulation.py::test_depuis_config_double_sens_par_defaut PASSED [ 21%]
tests/test_simulation.py::test_depuis_config_sens_unique PASSED          [ 23%]
tests/test_simulation.py::test_config_reelle PASSED                      [ 26%]
tests/test_simulation.py::test_generer_vehicule_nominal PASSED           [ 28%]
tests/test_simulation.py::test_generer_vehicule_une_seule_intersection PASSED [ 30%]
tests/test_simulation.py::test_generer_vehicule_reseau_sans_aucun_trajet_ne_boucle_pas PASSED [ 33%]
tests/test_simulation.py::test_vehicule_traverse_tout_le_trajet PASSED   [ 35%]
tests/test_simulation.py::test_depassement_conserve_a_l_intersection PASSED [ 38%]
tests/test_simulation.py::test_pas_de_temps_geant_position_reste_dans_la_route PASSED [ 40%]
tests/test_simulation.py::test_analyseur_vide PASSED                     [ 42%]
tests/test_simulation.py::test_analyseur_temps_parcours_sans_trajet PASSED [ 45%]
tests/test_simulation.py::test_analyseur_detecte_la_congestion PASSED    [ 47%]
tests/test_simulation.py::test_analyseur_zones_triees PASSED             [ 50%]
tests/test_simulation.py::test_simulation_nominale PASSED                [ 52%]
tests/test_simulation.py::test_simulation_zero_tour PASSED               [ 54%]
tests/test_simulation.py::test_n_tours_invalide_refuse[-1] PASSED        [ 57%]
tests/test_simulation.py::test_n_tours_invalide_refuse[2.5] PASSED       [ 59%]
tests/test_simulation.py::test_n_tours_invalide_refuse[10] PASSED        [ 61%]
tests/test_simulation.py::test_n_tours_invalide_refuse[None] PASSED      [ 64%]
tests/test_simulation.py::test_delta_t_non_positif_refuse[0] PASSED      [ 66%]
tests/test_simulation.py::test_delta_t_non_positif_refuse[-60] PASSED    [ 69%]
tests/test_simulation.py::test_delta_t_de_mauvais_type PASSED            [ 71%]
tests/test_simulation.py::test_fichier_config_inexistant PASSED          [ 73%]
tests/test_simulation.py::test_debit_negatif_refuse PASSED               [ 76%]
tests/test_simulation.py::test_plage_vitesse_incoherente_refusee PASSED  [ 78%]
tests/test_simulation.py::test_generation_selon_le_debit PASSED          [ 80%]
tests/test_simulation.py::test_debit_nul_aucun_vehicule PASSED           [ 83%]
tests/test_simulation.py::test_callback_tous_les_n_tours PASSED          [ 85%]
tests/test_simulation.py::test_conservation_des_vehicules PASSED         [ 88%]
tests/test_simulation.py::test_reproductible_avec_la_graine PASSED       [ 90%]
tests/test_simulation.py::test_graine_differente_resultat_different PASSED [ 92%]
tests/test_simulation.py::test_vehicules_evoluent_selon_les_regles_de_deplacement PASSED [ 95%]
tests/test_simulation.py::test_contraintes_du_reseau_respectees_a_chaque_tour PASSED [ 97%]
tests/test_simulation.py::test_les_trajets_utilisent_des_routes_existantes PASSED [100%]

============================== 42 passed in 0.32s ==============================
```

Figure 4.5: Tests du module `simulation`

```
$ python3 -m pytest -q

...........................................................................  [ 75%]
......................                                                       [100%]
95 passed in 0.33s
```

Figure 4.6: Résumé final

4.4 Absence de régression

Une simulation de 60 tours avec la configuration fournie donne exactement le même résumé qu'avant les corrections (36,09 km/h de vitesse moyenne, 2 867 trajets terminés, 733 véhicules en route, congestion sur `F-E` et `D-E`). Les corrections n'ont donc pas modifié le comportement sur des données valides.

Chapter 5

Anomalies détectées et corrections

Les anomalies sont de deux natures : des *validations absentes* (le code acceptait silencieusement des valeurs absurdes) et des *défauts de comportement*, plus graves (S1 et S5).

5.1 Défauts de comportement

N°	Où	Constat	Correction
S1	`ReseauRoutier.generer_vehicule`	**Boucle infinie** (`while True`) si aucun couple d'intersections n'est relié par un chemin	Nouvelle méthode `_tirer_trajet` : essais aléatoires bornés, puis parcours exhaustif des couples ; `ValueError` si aucun trajet
S5	`ReseauRoutier.mettre_a_jour`	Avec un grand pas de temps, un véhicule pouvait rester à une position supérieure à la longueur de sa route (le dépassement couvrant plusieurs routes courtes n'était traité qu'une fois)	Boucle `while` : le véhicule traverse autant de routes que nécessaire dans le même pas

Table 5.1: Anomalies de comportement

Listing 5.1: Correction de S5 : passage de plusieurs routes en un seul pas

```
for route, v in sortants:
    depassement = v.position - route.longueur
    while True:
        suivante = v.nom_route_suivante()
        if suivante is None: # fin de l'itinéraire
            route.retirer_vehicule(v)
            v.temps_arrivee = temps
            v.vitesse = 0.0
            self.vehicules_arrives.append(v)
            break
```

```
11          nouvelle = self.routes[suivante]
12          v.changer_de_route(nouvelle, depassement)
13          if v.position < nouvelle.longueur:
14              break # reste sur cette route
15          depassement = v.position - nouvelle.longueur
16          route = nouvelle
```

5.2 Validations manquantes

N°	Où	Constat (avant)	Comportement (après)
V1	`Vehicule()`	`vitesse_max` ≤ 0 accepté (2 tests)	`ValueError`
V2	`Vehicule()`	itinéraire vide accepté	`ValueError`
V3	`Vehicule()`	`temps_depart` négatif accepté	`ValueError`
V4	`Vehicule()`	identifiant vide accepté	`ValueError`
V5	`Vehicule()`	`vitesse_max="rapide"` non détecté à la création	`TypeError`
V6	`Vehicule.avancer`	`delta_t` négatif : le véhicule *reculait*	`ValueError`
V7	`Vehicule.avancer`	vitesse autorisée négative acceptée	`ValueError`
R1	`Route()`	longueur ≤ 0 acceptée (2 tests)	`ValueError`
R2	`Route()`	limite de vitesse ≤ 0 acceptée (2 tests) ; la division de Dijkstra pouvait planter	`ValueError`
R3	`Route()`	`nb_voies` $= 0$, -1 ou 1,5 accepté (3 tests)	`ValueError` / `TypeError`
R4	`Route()`	origine = destination acceptée	`ValueError`
R5	`Route()`	nom vide accepté	`ValueError`
R6	`Route()`	`longueur="longue"` : erreur obscure (`ValueError: could not convert...`)	`TypeError` explicite
R7	`Route.mettre_a_jour_vehicules`	`delta_t` négatif accepté	`ValueError`
S2	`plus_court_chemin`	intersection inconnue : résultat silencieux (`None` ou `[]`)	`ValueError`
S3	`ajouter_route`	route en double : l'ancienne était écrasée avec ses véhicules	`ValueError`
S4	`ajouter_intersection`	intersection en double écrasée	`ValueError`
S6	`lancer_simulation`	`n_tours` négatif : 0 tour exécuté sans erreur	`ValueError`
S7	`lancer_simulation`	`delta_t` ≤ 0 accepté (2 tests)	`ValueError`
S8	`Simulation()`	débit `vehicules_par_minute` négatif accepté	`ValueError`
S9	`Simulation()`	plage de vitesse `[130, 90]` acceptée	`ValueError`

Table 5.2: Validations ajoutées

Bilan. Ces anomalies correspondent aux **28 échecs** et au **test bloqué** du tableau avant correction (V : 8 échecs, R : 11, S : 9 échecs et 1 blocage). Toutes les validations passent par les trois fonctions de `_validation.py`.

Listing 5.2: Exemple de validation (`route.py`)

```
self.longueur = verifier_nombre("longueur", longueur, strictement_positif=True)
self.limite_vitesse = verifier_nombre("limite_vitesse", limite_vitesse,
                                      strictement_positif=True)
self.nb_voies = verifier_entier("nb_voies", nb_voies, minimum=1)
```

```
$ python3 -c "from simulateur import Route; Route('A-B','A','B',-100,50)"

Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from simulateur import Route; Route('A-B','A','B',-100,50)
                                  ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "/home/claude/tp2/simulateur_pkg/simulateur/route.py", line 53, in __init__
    self.longueur = verifier_nombre("longueur", longueur, strictement_positif=True)
                    ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/claude/tp2/simulateur_pkg/simulateur/_validation.py", line 32, in verifier_nombre
    raise ValueError(f"{nom} doit être strictement positif, reçu {valeur}")
ValueError: longueur doit être strictement positif, reçu -100.0
```

Figure 5.1: Message d'erreur clair après correction (`Route` avec longueur négative)

5.3 Choix de conception

- **Types** : un mauvais type lève `TypeError` ; une valeur hors domaine lève `ValueError`. Les booléens sont refusés comme nombres (sinon `True` passerait pour 1).
- Un test écrit trop strictement (`nb_voies=1.5` attendu en `ValueError`) a été corrigé : un décimal est un mauvais *type*, donc `TypeError`. Le code était correct, la spécification du test ne l'était pas.
- Les paramètres `n_tours` décimal ou texte levaient déjà une `TypeError` (par `range`) : ce n'était pas une anomalie.

Conclusion

Le package `simulateur` est désormais entièrement documenté (docstrings complètes, README, `pyproject.toml`) et validé par **95 tests pytest, tous réussis**. Écrire les tests à partir des spécifications avant de corriger a mis en évidence **29 problèmes** sur le code d'origine : 28 tests en échec et un test bloqué, soit deux défauts réels (une boucle infinie et un dépassement de route) et des validations absentes sur les paramètres. Le comportement sur des données valides est inchangé.

Pistes d'amélioration. mesurer la couverture avec `pytest-cov`, ajouter une intégration continue, et tester l'affichage graphique et l'export, hors du périmètre de ce TP.